# SOFTWARE DESIGN PRINCIPLES

## COMP6100001 - SOFTWARE ENGINEERING
## SESSION 08

UNIVERSITAS **BINA NUSANTARA**

SUBJECT MATTER EXPERT

**Kenny Jingga, S.Kom., M.T.**

# SESSION 08 – SOFTWARE DESIGN PRINCIPLES

## LEARNING OBJECTIVES

At the end of this session, students will be expected to be able to:

- Understand the Role and Importance of Software Design Principles
- Describe Key Software Design Principles, Including SOLID Principles
- Explore the Basics of Interface and Experience Design

# SESSION 08 – SOFTWARE DESIGN PRINCIPLES

## CONTENT OUTLINE

- Introduction to design principles
- SOLID principles
- Applying principles to scenarios

# INTRODUCTION TO SOFTWARE DESIGN PRINCIPLES

## WHAT IS SOFTWARE DESIGN?

- **Definition**:
  - Software design is the process of defining the architecture, components, interfaces, and other characteristics of a system or component.
- **Purpose**:
  - Transform software requirements into a blueprint for constructing the software.
  - Ensure that the software system meets specified requirements.
- **Key Points**:
  - Involves problem-solving and planning a software solution.
  - Aims to produce a model or representation of the software system.

## USER INTERFACE (UI) DESIGN

- **Definition**:
  - The design of the interface through which users interact with software applications.
- **Principles**:
  - **Clarity**: Interfaces should be clear and unambiguous.
  - **Consistency**: Uniform design elements across the application.
  - **Feedback**: Provide users with feedback on their actions.
  - **Affordance**: Design elements should suggest their functionality.

## USER EXPERIENCE (UX) DESIGN

- **Definition**:
  - The process of enhancing user satisfaction by improving usability and accessibility.
- **Principles**:
  - **User-Centered Desi**gn: Focus on users' needs and preferences.
  - **Simplicity**: Streamline user tasks to be as straightforward as possible.
  - **Accessibility**: Ensure the interface is usable by people with varying abilities.

Difference between

# USER INTERFACE & USER EXPERIENCE

# INTERFACE AND EXPERIENCE DESIGN

## TYPICAL DESIGN ERROR

**Easy to learn?**
**Easy to use?**
**Easy to understand?**

- lack of consistency
- too much memorization
- no guidance / help
- no context sensitivity
- poor response
- Arcane/unfriendly

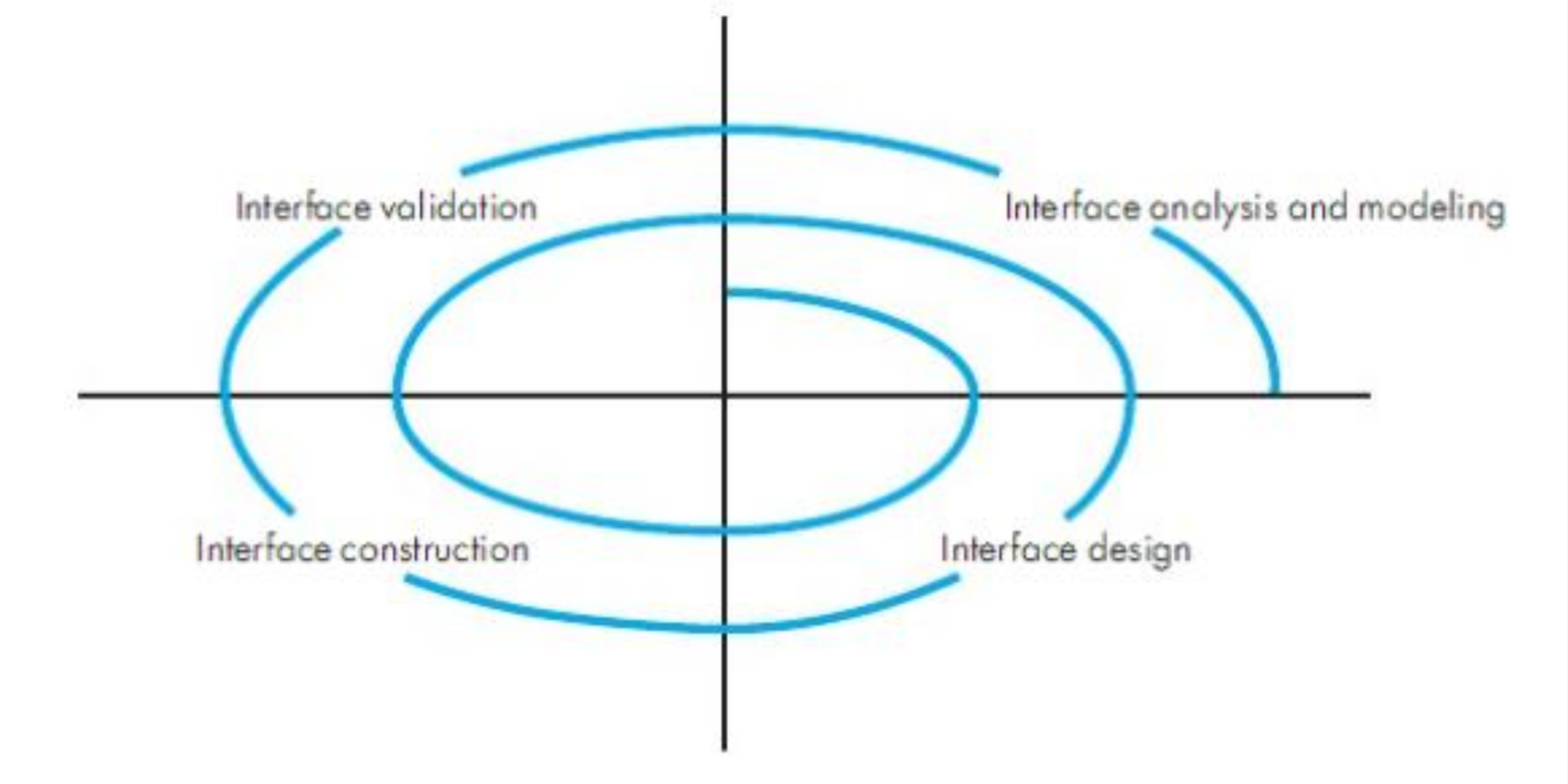
## EIGHT GOLDEN RULES OF INTERFACE DESIGN

- Strive for consistency
- Enable frequent users to use shortcuts
- Offer informative feedback
- Design dialogue to yield closure
- Offer simple error handling
- Permit easy reversal of actions
- Support internal locus of control
- Reduce short-term memory load

https://www.interaction-design.org/literature/article/shneiderman-s-eight-golden-rules-will-help-you-design-better-interfaces

# INTERFACE AND EXPERIENCE DESIGN

## THE USER INTERFACE DESIGN PROCESS



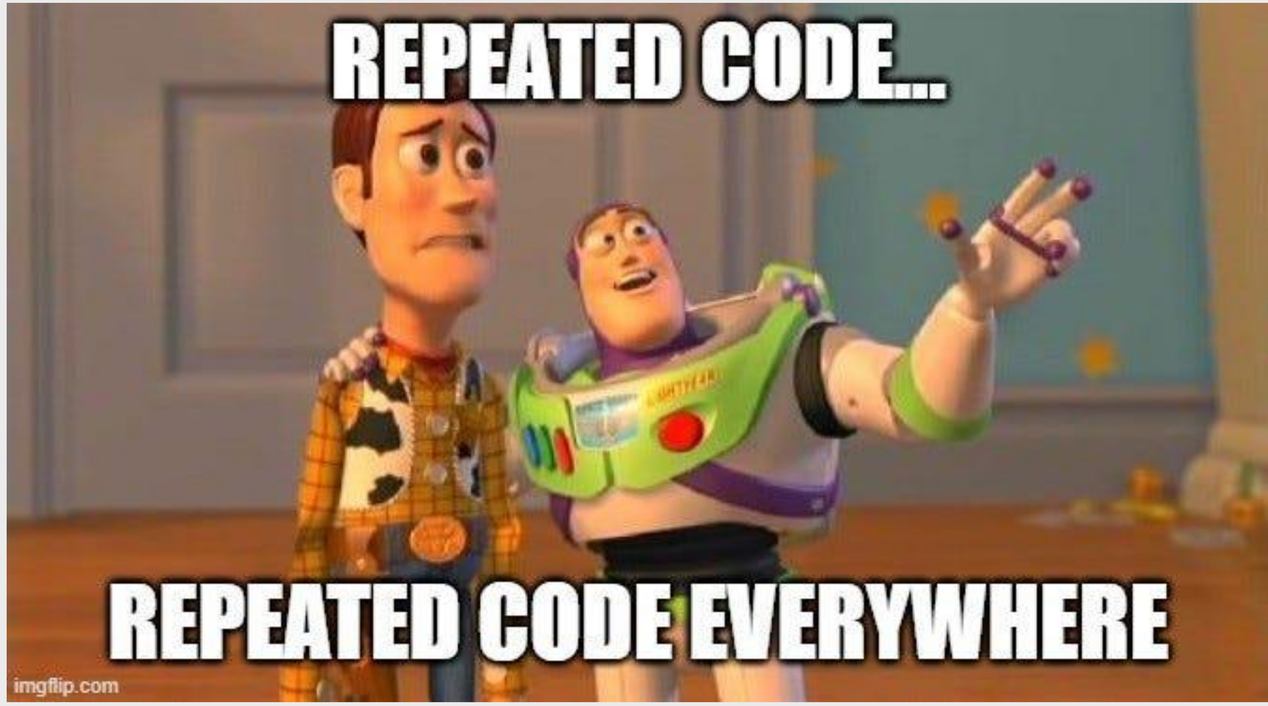
## DESIGN ISSUES

- Response time
- Help facilities
- Error handling
- Menu and command labeling
- Application accessibility
- Internationalization

# FUNDAMENTAL DESIGN PRINCIPLES

## DRY (DON'T REPEAT YOURSELF)

- **Definition**:
  - Avoid duplication of code or functionality.
- **Benefits**:
  - Reduces maintenance efforts.
  - Minimizes bugs due to inconsistent changes.
- **Example**:
  - Using functions or classes to encapsulate reusable code.



# FUNDAMENTAL DESIGN PRINCIPLES

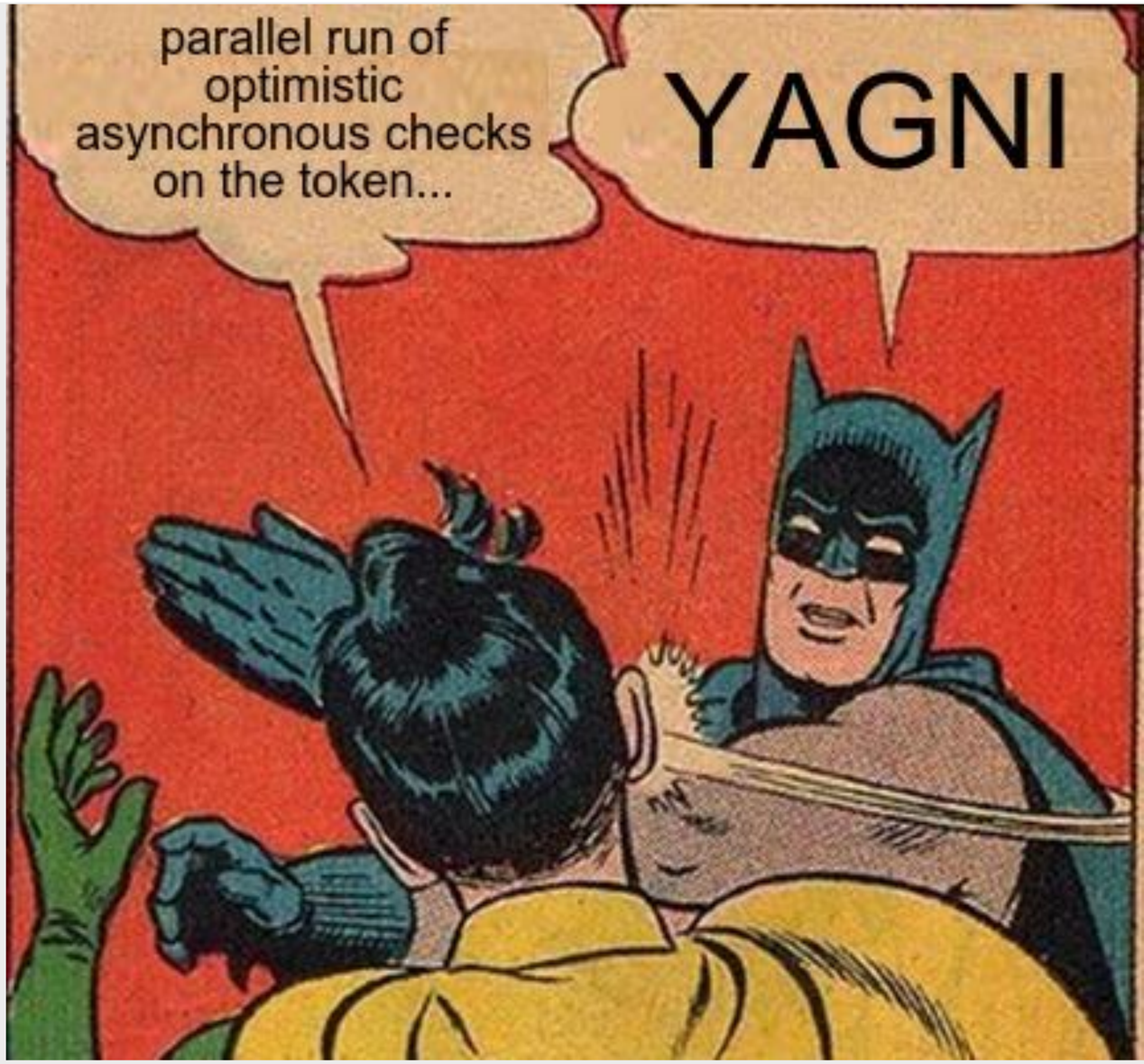
## KISS (KEEP IT SIMPLE, STUPID)

- **Definition**:
  - Simplicity should be a key goal; unnecessary complexity should be avoided.
- **Benefits**:
  - Enhances readability and understandability.
  - Simplifies debugging and maintenance.
- **Example**:
  - Choosing simple algorithms over complex ones when appropriate.

# FUNDAMENTAL DESIGN PRINCIPLES

## YAGNI (YOU AREN'T GONNA NEED IT)

- **Definition**:
  - Do not implement functionality until it is necessary.
- **Benefits**:
  - Saves time and resources.
  - Prevents code bloat.
- **Example**:
  - Avoid adding features based on speculative future needs.



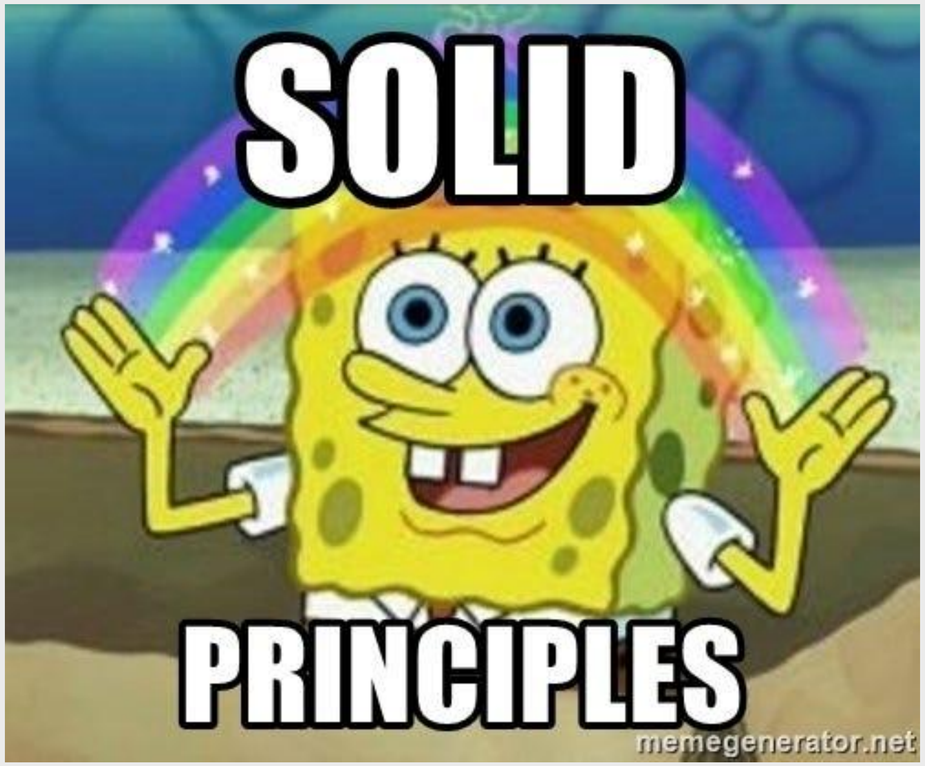
# SOLID PRINCIPLES

## OVERVIEW

- A set of five principles aimed at making object-oriented designs more understandable, flexible, and maintainable.

**The Principles**:

- Single Responsibility Principle (SRP)
- Open-Closed Principle (OCP)
- Liskov Substitution Principle (LSP)
- Interface Segregation Principle (ISP)
- Dependency Inversion Principle (DIP)



# SOLID PRINCIPLES

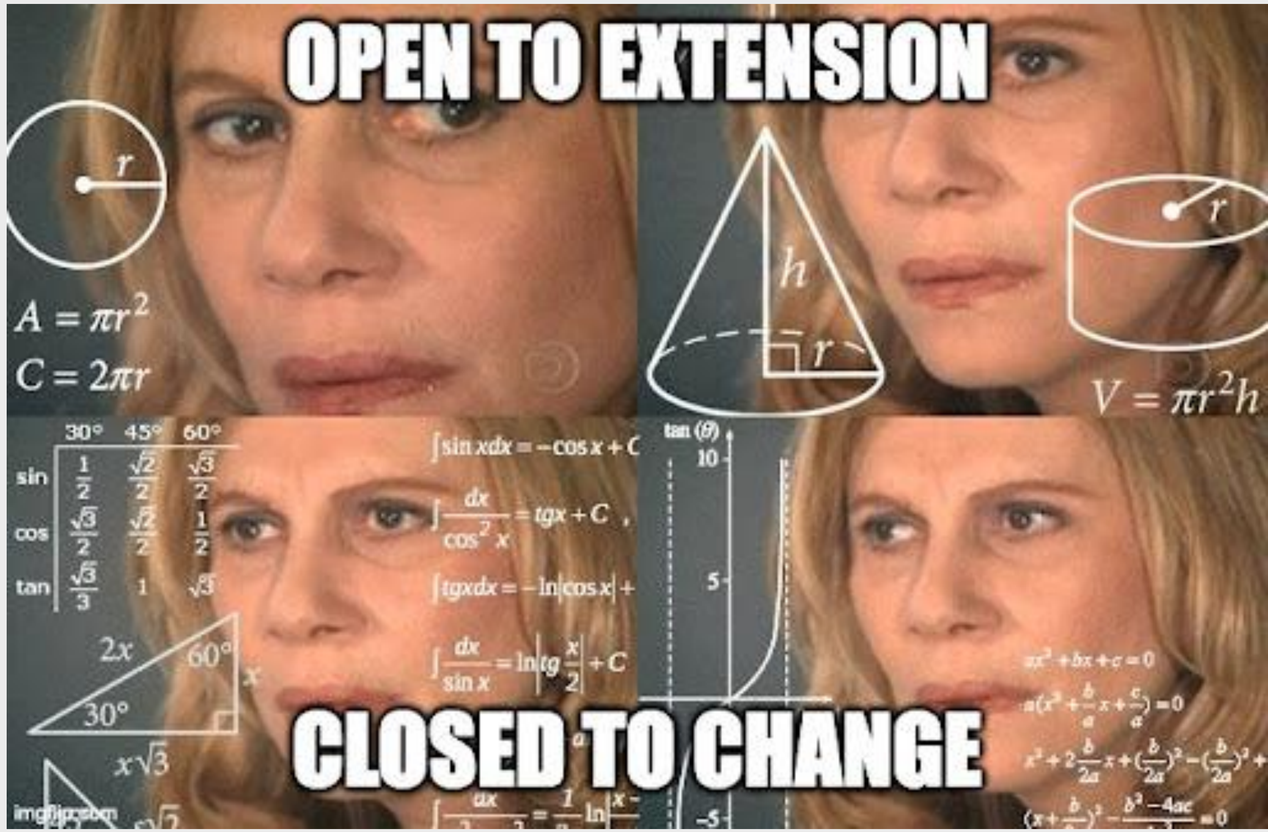
## SINGLE RESPONSIBILITY PRINCIPLE (SRP)

- **Definition**:
  - A class should have one, and only one, reason to change.
- **Benefits**:
  - Increases cohesion.
  - Simplifies testing and maintenance.
- **Example**:
  - Separating data handling from user interface logic.

# SOLID PRINCIPLES

## OPEN-CLOSED PRINCIPLE (OCP)

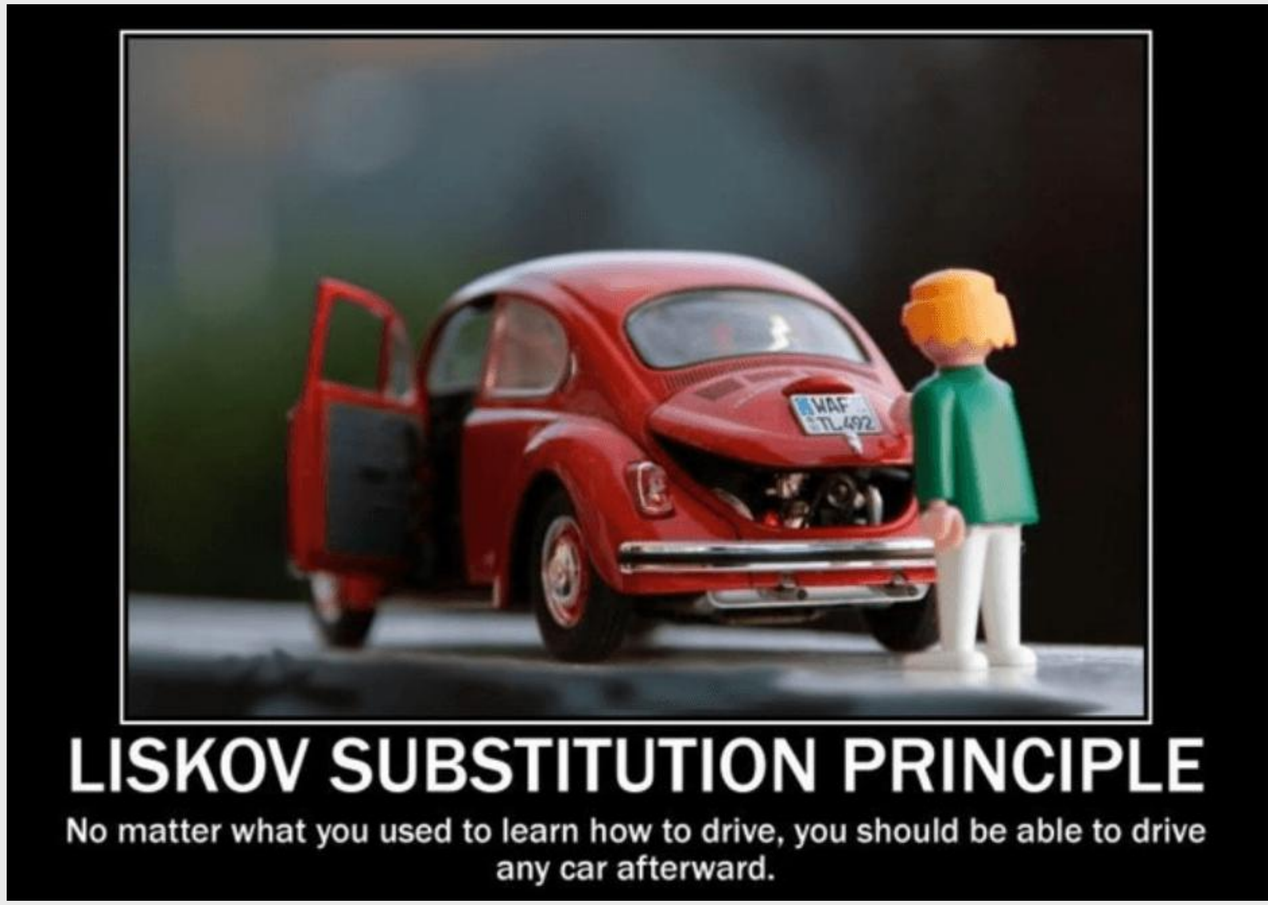
- **Definition**:
  - Software entities should be open for extension but closed for modification.
- **Benefits**:
  - Allows adding new features without changing existing code.
  - Reduces the risk of introducing new bugs.
- **Example**:
  - Using abstract classes or interfaces to allow new implementations.



# SOLID PRINCIPLES

## LISKOV SUBSTITUTION PRINCIPLE (LSP)

- **Definition**:
  - Subtypes must be substitutable for their base types.
- **Benefits**:
  - Ensures that a subclass can stand in for its superclass without unexpected behavior.
- **Example**:
  - If class *Square* inherits from *Rectangle*, it should not alter expected behaviors.



# SOLID PRINCIPLES

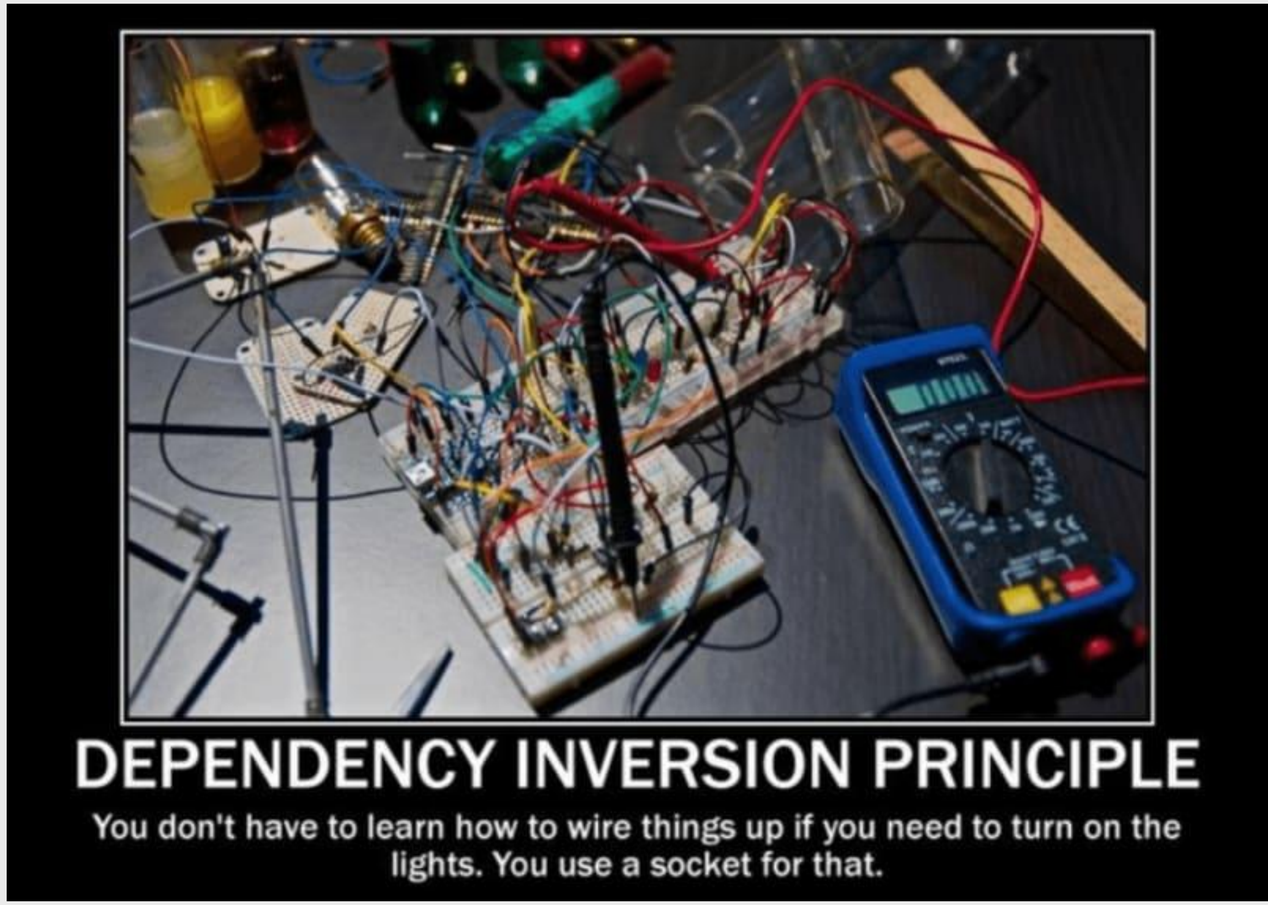
## INTERFACE SEGREGATION PRINCIPLE (ISP)

- **Definition**:
  - Clients should not be forced to depend on interfaces they do not use.
- **Benefits**:
  - Promotes decoupling.
  - Reduces the impact of changes.
- **Example**:
  - Splitting a large interface into smaller, more specific ones.

# SOLID PRINCIPLES

## DEPENDENCY INVERSION PRINCIPLE (DIP)

- **Definition**:
  - High-level modules should not depend on low-level modules; both should depend on abstraction.
- **Benefits**:
  - Enhances modularity and reusability.
  - Simplifies testing through dependency injection.
- **Example**:
  - Depending on interfaces rather than concrete classes.



# APPLYING DESIGN PRINCIPLES TO REAL-WORLD SCENARIOS

## CASE STUDY: ONLINE SHOPPING PLATFORM

- **Scenario**:
  - Building an e-commerce website with user accounts, product listings, shopping carts, and order processing.
- **Application of Principles**:
  - **SRP**: Separate classes for User Management, Product Catalog, Shopping Cart, and Order Processing.
  - **OCP**: Use interfaces for payment processing to allow adding new payment methods without altering existing code.
  - **DRY**: Centralize input validation logic to avoid repetition.
  - **KISS**: Simplify the checkout process to require minimal user steps.

People
Innovation
Excellence

# REFERENCES


- Roger Pressman and Bruce Maxim. (2020). Software Engineering: A Practitioner's Approach. 9th. McGraw Hill. New York. ISBN: 978-1259872976.